



Descubrimiento Evolutivo de Estrategias Cuantitativas en Mercados de Criptomonedas

*Evolución Gramatical, Calidad-Diversidad
y Validación Estadística*

Juan Manuel Targa

Universidad del CEMA

Marzo 2026

Índice

1. Introducción	3
1.1. Contribución	4
2. Marco Teórico	5
2.1. Algoritmos evolutivos: la idea central	5
2.2. De los Algoritmos Genéticos a la Programación Genética	5
2.3. Evolución Gramatical: la solución elegante	6
2.4. ¿Por qué no fuerza bruta?	8
2.5. MAP-Elites: diversidad como objetivo	9
2.6. Métricas de evaluación	9
2.7. Validación estadística	10
2.7.1. Validación Cruzada Combinatoria Purgada (CPCV)	10
2.7.2. Probabilidad de Sobreajuste de Backtest (PBO)	11
2.7.3. Ratio de Sharpe Deflactado (DSR)	11
2.7.4. Test de permutación de señales	12
3. Trabajos Relacionados	12
4. Metodología	13
4.1. La gramática de estrategias	13
4.1.1. Indicadores: dos familias tipadas	13
4.1.2. Parámetros de salida	14
4.2. Evaluación vectorizada	14
4.3. Motor evolutivo	14
4.4. Refinamiento local con CMA-ES	15
4.5. Pipeline de validación	16
4.6. Validación walk-forward	17
4.7. Evaluación final sobre datos reservados	17
5. Configuración Experimental	17
5.1. Datos	17
5.2. Parámetros de evolución	18
5.3. Ejecuciones	18
6. Resultados	18
6.1. Evolución	18
6.2. Estrategias descubiertas	19
6.3. Validación estadística	20
6.4. Construcción de portafolios	21

6.5. Extensión multi-activo	23
6.6. Estabilidad walk-forward	24
7. Discusión	25
7.1. El rol de la diversificación	25
7.2. Interpretabilidad	25
7.3. Eficiencia de mercado y capacidad predictiva individual	25
7.4. La asimetría CMA-ES entre activos	26
7.5. Limitaciones	26
8. Conclusiones y Trabajo Futuro	27
8.1. Conclusiones	27
8.2. Trabajo futuro	27

Resumen

Este trabajo desarrolla un framework para descubrir automáticamente reglas cuantitativas de entrada y salida en mercados de criptomonedas, utilizando datos intradía de 15 minutos. El enfoque combina Evolución Gramatical —una técnica de computación evolutiva que genera expresiones válidas a partir de una gramática formal— con MAP-Elites, un algoritmo de calidad-diversidad que mantiene un repertorio de soluciones variadas en lugar de converger a una única “mejor” respuesta. Una segunda fase de optimización local con CMA-ES refina los parámetros continuos de las estructuras descubiertas. La validación emplea Validación Cruzada Combinatoria Purgada, análisis de Probabilidad de Sobreajuste, tests de permutación de señales y validación walk-forward sobre múltiples ventanas temporales, seguidos de una evaluación final sobre datos reservados.

Se realizaron experimentos sobre tres activos (BTC/USDT, ETH/USDT, BNB/USDT) con 84.672 observaciones por activo (enero 2023 a mayo 2025). Del proceso de evolución y validación emergieron 87 estrategias que superaron los tres criterios estadísticos en BTC, y se obtuvieron resultados comparables en ETH y BNB. La optimización CMA-ES mejoró significativamente las estrategias de ETH y BNB, aunque no las de BTC —un hallazgo que sugiere diferencias en la rugosidad del paisaje de aptitud entre activos. Un portafolio diversificado de 10 estrategias decorrelacionadas distribuidas entre los tres activos demostró consistencia en validación walk-forward: 9 de 10 ventanas positivas en BTC y ETH, 8 de 10 en BNB. En el período de evaluación final (junio–noviembre 2025), el portafolio obtuvo un retorno anualizado del 9,9% con un drawdown máximo de $-0,8\%$, en un período en que BTC perdió el $31,6\%$ de su valor.

Los resultados sugieren que, si bien las estrategias individuales basadas en indicadores técnicos muestran capacidad predictiva limitada cuando se las evalúa con tests de significancia estrictos, la combinación de múltiples reglas decorrelacionadas a través de varios activos puede generar perfiles de riesgo-retorno favorables incluso en condiciones adversas de mercado.

1. Introducción

Desde la aparición de Bitcoin en 2008 [13], los mercados de criptomonedas operan de forma continua (24 horas, 7 días) con niveles de volatilidad que superan ampliamente a los mercados financieros tradicionales. Esta combinación de acceso permanente y alta variabilidad plantea una pregunta natural: ¿existen regularidades en estos mercados que puedan ser identificadas de forma sistemática?

La pregunta tiene raíces profundas. La **Hipótesis de Mercados Eficientes** [6] sostiene que los precios reflejan toda la información disponible, lo que implica que los patrones históricos de precios no deberían contener información útil para predecir movimientos

futuros. Si esto es cierto, cualquier regla basada en datos pasados debería tener un rendimiento esperado igual al del mercado una vez descontados los costos de transacción.

Sin embargo, la teoría admite grados. Fama distingue tres formas de eficiencia: débil (los precios pasados no predicen los futuros), semifuerte (toda la información pública está incorporada) y fuerte (incluso la información privada está reflejada). Los mercados de criptomonedas, por su relativa juventud y la heterogeneidad de sus participantes, podrían exhibir ineficiencias temporales que un sistema automatizado podría detectar.

El problema es que buscar patrones en datos históricos es inherentemente peligroso. Cuando se evalúan suficientes reglas sobre los mismos datos, alguna inevitablemente mostrará rendimientos positivos *por azar*. Este fenómeno, conocido como **sesgo de selección múltiple** o *data snooping* [20], es el obstáculo central de las finanzas cuantitativas. Una regla que memoriza las particularidades de un período histórico pero no captura una regularidad genuina se dice **sobreajustada** (*overfitted*): parece funcionar en retrospectiva pero fracasa cuando se aplica a datos nuevos.

1.1. Contribución

El enfoque convencional para encontrar reglas cuantitativas es la búsqueda exhaustiva o el barrido de parámetros (*grid search*): evaluar todas las combinaciones posibles de indicadores y umbrales, y quedarse con las que mejor funcionaron históricamente. Este enfoque tiene dos defectos fundamentales: es computacionalmente inviable cuando el espacio de búsqueda es grande (como en nuestro caso, con $\sim 10^8$ combinaciones), y es estadísticamente peligroso —cuantas más reglas se evalúan, mayor la probabilidad de encontrar una que funcione por azar.

Este trabajo propone un framework que aborda ambos problemas:

1. **Un motor de búsqueda basado en Evolución Gramatical**, que explora el espacio de reglas cuantitativas de forma inteligente: guiada por la aptitud de las soluciones ya encontradas, en lugar de recorrerlo uniformemente. Evoluciona simultáneamente la estructura de las reglas y sus parámetros numéricos, y una segunda fase de optimización local (CMA-ES) refina los parámetros continuos.
2. **Un mecanismo de diversidad basado en MAP-Elites**, que previene la convergencia prematura manteniendo un mapa de soluciones organizadas por comportamiento, no solo por calidad. Esto produce los ingredientes necesarios para construir portafolios decorrelacionados.
3. **Un pipeline de validación estadística en tres capas**: protecciones anti-sobreajuste durante la evolución (rotación de ventanas temporales), validación post-evolución

con múltiples tests (CPCV, PBO, permutación de señales, walk-forward), y una evaluación final sobre datos reservados.

El resultado principal es un portafolio multi-activo de estrategias decorrelacionadas que muestra resiliencia en un mercado bajista, junto con un análisis honesto de los alcances y limitaciones de los patrones descubiertos —incluyendo la observación de que ninguna estrategia individual alcanza significancia estadística según el test de White, lo cual refuerza la importancia de la diversificación.

2. Marco Teórico

2.1. Algoritmos evolutivos: la idea central

Los **algoritmos evolutivos** [7, 9] son métodos de optimización inspirados en la selección natural. La analogía con la biología es útil para entender su funcionamiento.

Imaginemos una población de individuos, donde cada individuo es una solución candidata a un problema. Algunos individuos resuelven el problema mejor que otros —tienen mayor *aptitud* (fitness)—. El algoritmo imita tres procesos biológicos:

- **Selección:** los individuos más aptos tienen mayor probabilidad de reproducirse. Esto no significa que los mejores siempre sobrevivan —hay estocasticidad, como en la naturaleza— pero el sesgo estadístico favorece a los más aptos a lo largo de muchas generaciones.
- **Recombinación** (crossover): dos individuos “padres” combinan partes de su información genética para producir “hijos” que heredan características de ambos. En la práctica, esto significa tomar la primera mitad de una solución y la segunda mitad de otra.
- **Mutación:** con baja probabilidad, se introducen cambios aleatorios pequeños en un individuo. La mutación es el mecanismo que introduce novedad: sin ella, la población solo podría recombinar lo que ya existe, sin explorar regiones nuevas del espacio de búsqueda.

El ciclo se repite: evaluar aptitud, seleccionar, recombinar, mutar, y volver a evaluar. Con el tiempo, la población tiende a mejorar. Esto no requiere ningún conocimiento sobre *cómo* resolver el problema —solo una forma de medir qué tan buena es cada solución.

2.2. De los Algoritmos Genéticos a la Programación Genética

En un **Algoritmo Genético** (AG) clásico, cada individuo es una cadena de números o bits que codifican una solución. Por ejemplo, los parámetros de un modelo. El AG optimiza

estos números.

La **Programación Genética** (GP) [10] da un paso más ambicioso: en lugar de optimizar parámetros, optimiza *programas enteros*. Cada individuo no es un conjunto de números, sino una expresión simbólica —un árbol donde los nodos internos son operadores (suma, comparación, AND lógico) y las hojas son variables o constantes.

Por ejemplo, una regla como “comprar cuando el RSI está por debajo de 30 y el volumen supera el promedio” se representaría como un árbol con un nodo AND en la raíz, un nodo “<” a la izquierda (comparando RSI con 30) y otro nodo “>” a la derecha (comparando volumen con su media).

GP es poderosa pero tiene problemas prácticos:

- Los árboles tienden a crecer sin control (*bloat*) sin que su calidad mejore proporcionalmente.
- El crossover entre árboles —intercambiar subárboles entre dos padres— frecuentemente destruye soluciones buenas, porque los subárboles intercambiados pueden ser incompatibles entre sí.
- Es difícil garantizar que las expresiones generadas sean *válidas*: ¿qué significa comparar un precio en dólares con un volumen en BTC?

2.3. Evolución Gramatical: la solución elegante

La **Evolución Gramatical** (GE) [14] resuelve estos problemas con una idea simple pero profunda: **separar completamente la representación genética de la expresión que produce**.

En GE, cada individuo es simplemente un vector de números enteros, llamados **codones**. Por ejemplo:

$$\text{genoma} = [42, 100, 200, 17, 55, 83, 12, \dots]$$

Este vector no tiene significado intrínseco. Para convertirlo en una expresión útil, se usa una **gramática BNF** (Backus-Naur Form) que define las reglas de producción del lenguaje de estrategias. El proceso de decodificación funciona así:

1. Se parte de un símbolo inicial: <estrategia>.
2. Este símbolo tiene varias formas posibles de expandirse (producciones). Si hay k opciones, se lee el siguiente codón del genoma y se calcula $\text{codón} \bmod k$ para elegir cuál usar.

3. La producción elegida puede contener nuevos símbolos no resueltos. Se repite el proceso para cada uno, consumiendo codones secuencialmente.
4. El proceso termina cuando todos los símbolos están resueltos (son terminales).

¿Por qué esto es elegante? Por tres razones:

1. **Validez garantizada:** la gramática define qué expresiones son posibles. Si la gramática está bien diseñada, *cualquier* secuencia de enteros produce una expresión válida (o ninguna, si los codones se agotan —pero nunca una expresión inválida).
2. **Operadores genéticos triviales:** como el genoma es un vector de enteros, el crossover es simplemente cortar dos vectores en un punto e intercambiar las mitades. La mutación es sumar o restar 1 a un codón. Estos operadores *siempre* producen genomas válidos, porque la validez la garantiza la gramática, no el genoma.
3. **Estructura y parámetros co-evolucionan:** un mismo genoma codifica tanto *qué* indicadores usar (estructura) como *con qué parámetros* (períodos, umbrales). No hay que optimizar uno y fijar el otro; todo evoluciona junto.

Ejemplo concreto. Supongamos que la gramática define:

```
<estrategia> ::= <direccion> WHEN <regla> EXIT <salidas>
<direccion>  ::= LONG | SHORT
<regla>      ::= <condicion>
                | <condicion> AND <condicion>
<condicion>  ::= <indicador> > <umbral>
                | <indicador> < <umbral>
<indicador>  ::= RSI(<periodo>) | STOCHASTIC(<periodo>)
<periodo>    ::= 7 | 14 | 21
<umbral>     ::= 20 | 50 | 80
```

Con el genoma [5, 1, 0, 2, 1, 0, 2, ...]:

1. <estrategia>: 1 producción → <dirección> WHEN <regla> EXIT <salidas>.
2. <dirección>: 2 opciones, 5 mód 2 = 1 → SHORT.
3. <regla>: 2 opciones, 1 mód 2 = 1 → <condición> AND <condición>.
4. Primera <condición>: 2 opciones, 0 mód 2 = 0 → <indicador> > <umbral>.
5. <indicador>: 2 opciones, 2 mód 2 = 0 → RSI(<período>).
6. <período>: 3 opciones, 1 mód 3 = 1 → 14.

7. `<umbral>`: 3 opciones, $0 \bmod 3 = 0 \rightarrow 20$.

8. Segunda condición se resuelve de forma análoga.

Resultado: `SHORT WHEN RSI(14) > 20 AND STOCHASTIC(7) < 80 EXIT ...`

Un cambio mínimo en un codón —por ejemplo, cambiar el 5 inicial por 4— cambiaría la dirección de SHORT a LONG. Cambiar el 1 del período de 14 a 0 cambiaría el período a 7. Así, mutaciones pequeñas en los enteros producen variaciones graduales en las estrategias, exactamente lo que necesitamos para una búsqueda evolutiva eficiente.

2.4. ¿Por qué no fuerza bruta?

Antes de describir MAP-Elites, vale la pena detenerse en una pregunta natural: ¿por qué no evaluar simplemente *todas* las estrategias posibles?

El espacio de búsqueda definido por nuestra gramática contiene del orden de 10^8 estrategias distintas. Evaluar cada una sobre 85.000 observaciones toma aproximadamente 1 milisegundo gracias a la evaluación vectorizada, lo que implica que una enumeración exhaustiva tomaría del orden de 10^5 segundos —unas 28 horas. Parece factible, pero hay un problema más profundo: no se trata solo de evaluar, sino de *validar*. Cada estrategia candidata requiere validación cruzada CPCV (252 evaluaciones), test de permutación (1.000 repeticiones) y verificación de múltiples métricas. El costo real de evaluar rigurosamente una estrategia es al menos 1.000 veces mayor que una evaluación simple, lo que lleva la enumeración exhaustiva a ~ 3 años de cómputo.

Más importante aún, la fuerza bruta trata todas las regiones del espacio por igual: invierte el mismo esfuerzo en estrategias claramente inviables (un RSI comparado con un umbral absurdo) que en regiones prometedoras. Un muestreo aleatorio uniforme sufre el mismo problema —la mayoría de las muestras caen en regiones de baja aptitud.

La búsqueda evolutiva resuelve esto mediante **explotación guiada por aptitud**: los individuos más aptos se reproducen más, concentrando la exploración en las regiones que ya mostraron señales de calidad. Simultáneamente, la mutación y MAP-Elites garantizan que no se abandone la exploración de regiones distantes. El resultado es un balance entre *exploración* (descubrir regiones nuevas) y *explotación* (profundizar en las prometedoras) que ningún método de muestreo uniforme puede lograr.

En términos concretos: con una población de 300 individuos durante 100 generaciones, el sistema evalúa ~ 30.000 estrategias —el 0,03 % del espacio total— pero guiadas hacia regiones de alta calidad. Como veremos en la sección de resultados, este presupuesto modesto es suficiente para descubrir decenas de estrategias que superan validación estadística rigurosa.

2.5. MAP-Elites: diversidad como objetivo

Un problema clásico de los algoritmos evolutivos es la **convergencia prematura**: la población entera converge hacia una misma solución antes de haber explorado suficientemente el espacio de búsqueda. Esto es problemático porque la mejor solución encontrada tempranamente puede ser un *óptimo local* —buena en su vecindad, pero inferior a soluciones que nunca se exploraron.

MAP-Elites [12] propone una solución radical: en lugar de buscar *la* mejor solución, buscar *la mejor solución de cada tipo*. Para ello, se define un espacio de comportamientos posibles como una grilla multidimensional, donde cada celda representa un “nicho” comportamental.

En nuestro caso, definimos tres dimensiones:

- **Frecuencia**: cuántas operaciones genera la estrategia por mes. Baja (≤ 5), media (5–20) o alta (> 20).
- **Complejidad**: cuántas condiciones tiene la regla de entrada (1, 2, 3, 4, o 5+).
- **Régimen dominante**: en qué tipo de mercado rinde mejor (alcista, bajista o lateral).

La grilla tiene $3 \times 5 \times 3 = 45$ celdas. MAP-Elites mantiene, para cada celda, la mejor estrategia encontrada que exhibe ese comportamiento específico. Por ejemplo: “la mejor estrategia de alta frecuencia, con 2 condiciones, que rinde bien en mercado bajista”.

¿Por qué es útil? Porque garantiza que la población final contiene estrategias *diversas*: unas operan mucho y otras poco, unas son simples y otras complejas, unas funcionan en mercados alcistas y otras en bajistas. Esta diversidad resulta crucial para construir portafolios, donde la decorrelación entre estrategias reduce el riesgo del conjunto.

2.6. Métricas de evaluación

Para medir la calidad de una estrategia, utilizamos varias métricas que capturan diferentes aspectos del rendimiento.

Ratio de Sortino. Mide el retorno obtenido por cada unidad de riesgo a la baja [18]:

$$\text{Sortino} = \frac{\bar{r} \cdot P}{\sigma_d \cdot \sqrt{P}} \quad (1)$$

donde $\bar{r}$ es el retorno medio por período, σ_d es la desviación estándar calculada *solo sobre los retornos negativos*, y P es el número de períodos por año (35.040 para velas de 15 minutos). La diferencia con el ratio de Sharpe [17] es importante: el Sharpe penaliza *toda*

la volatilidad, incluyendo la volatilidad alcista, que para un inversor no es riesgo sino oportunidad. El Sortino solo penaliza las caídas.

Ratio de Calmar. Relaciona el crecimiento anualizado con la peor caída experimentada [21]:

$$\text{Calmar} = \frac{\text{CAGR}}{|\text{Max Drawdown}|} \quad (2)$$

CAGR (Tasa de Crecimiento Anual Compuesta).

$$\text{CAGR} = \left(\frac{V_f}{V_i} \right)^{P/n} - 1 \quad (3)$$

donde V_f y V_i son los valores final e inicial, n el número de períodos observados y P los períodos por año.

Drawdown máximo. La mayor caída porcentual desde un máximo hasta el siguiente mínimo:

$$\text{Max DD} = \min_t \left\{ \frac{V_t - \max_{s \leq t} V_s}{\max_{s \leq t} V_s} \right\} \quad (4)$$

Factor de beneficio. La relación entre lo ganado y lo perdido:

$$\text{PF} = \frac{\sum \text{ganancias}}{\sum |\text{pérdidas}|} \quad (5)$$

Un PF mayor a 1 indica que la estrategia gana más de lo que pierde en total.

2.7. Validación estadística

Evaluar muchas estrategias sobre los mismos datos genera un sesgo inevitable: la mejor va a parecer buena *simplemente porque evaluamos muchas*. Los tests estadísticos que empleamos están diseñados específicamente para detectar y corregir este sesgo.

2.7.1. Validación Cruzada Combinatoria Purgada (CPCV)

En machine learning, la validación cruzada divide los datos en partes, entrena en unas y evalúa en otras. Pero en finanzas hay un problema adicional: los datos son *temporales* y están autocorrelacionados. Si una observación de las 14:00 está en el conjunto de entrenamiento y una de las 14:15 está en el de evaluación, la cercanía temporal contamina la evaluación.

La CPCV [2] resuelve esto en tres pasos:

1. Divide los datos en N bloques contiguos en el tiempo (usamos $N = 10$).

2. Genera todas las combinaciones posibles de bloques para entrenamiento y evaluación. Con $N = 10$ y mitad para cada uno, hay $\binom{10}{5} = 252$ combinaciones.
3. Para cada combinación, elimina (*purga*) las observaciones cercanas a la frontera temporal entre entrenamiento y evaluación, y añade un período de *embargo* adicional como margen de seguridad.

El resultado no es un número, sino una *distribución* de 252 rendimientos fuera de muestra, mucho más informativa que una única evaluación.

2.7.2. Probabilidad de Sobreajuste de Backtest (PBO)

A partir de la distribución generada por CPCV, la PBO [3] calcula qué fracción de las evaluaciones tuvo rendimiento negativo:

$$\text{PBO} = \frac{\#\{\text{splits con Sortino} \leq 0\}}{\text{total de splits}} \quad (6)$$

La interpretación es directa: si una estrategia tiene un patrón genuino, debería rendir positivamente en la mayoría de los períodos de evaluación. Un PBO de 0,10 significa que solo el 10 % de las evaluaciones fueron negativas —señal fuerte. Un PBO de 0,60 significa que más de la mitad fueron negativas —probable sobreajuste. Usamos $\text{PBO} < 0,50$ como criterio de aprobación.

2.7.3. Ratio de Sharpe Deflactado (DSR)

El DSR [1] responde a la pregunta: “dado que evaluamos n estrategias, ¿la mejor es genuinamente buena o es la ganadora esperable por azar?”

Si evaluamos n estrategias y ninguna tiene capacidad predictiva real, la mejor tendrá un Sharpe esperado de:

$$\mathbb{E}[\text{máx}(\widehat{SR})] \approx \sigma_{SR} \left[\sqrt{2 \ln n} - \frac{\ln \pi + \ln(\ln n)}{2\sqrt{2 \ln n}} \right] \quad (7)$$

donde $\sigma_{SR} = \sqrt{P/(T-1)}$ es el error estándar del Sharpe, T el número de observaciones y P los períodos por año.

El DSR compara el Sharpe observado contra este umbral:

$$\text{DSR} = \Phi \left(\frac{\widehat{SR} - \mathbb{E}[\text{máx}(\widehat{SR})]}{\sigma_{SR}} \right) \quad (8)$$

donde Φ es la función de distribución acumulada normal. Un DSR cercano a 1 indica que

el Sharpe observado supera lo esperable por azar; cercano a 0, que probablemente es un artefacto de la búsqueda.

2.7.4. Test de permutación de señales

Este test evalúa directamente si *cuándo* la estrategia genera señales importa, o si cualquier momento habría sido igual:

1. Se ejecuta la estrategia y se registran los momentos exactos en que genera señales de entrada.
2. Se ejecuta el backtest y se calcula el Sortino real.
3. Se permutan aleatoriamente las posiciones de las señales —manteniendo la misma cantidad total— y se recalcula el Sortino con las señales reorganizadas.
4. Se repite 1.000 veces.
5. El p -valor es la fracción de permutaciones que igualan o superan al Sortino real.

Si $p < 0,05$, los momentos de entrada elegidos por la estrategia son significativamente mejores que el azar, lo cual sugiere que la combinación de indicadores captura información genuina sobre el estado del mercado.

3. Trabajos Relacionados

La aplicación de computación evolutiva a mercados financieros tiene una historia extensa. Chen *et al.* [5] demostraron la viabilidad de Programación Genética para descubrir reglas en mercados accionarios. Potvin *et al.* [15] extendieron el enfoque con operadores especializados. Brabazon y O'Neill [4] fueron pioneros en aplicar Evolución Gramatical al dominio financiero, explotando la capacidad de GE para evolucionar estructura y parámetros simultáneamente.

Trabajos recientes incorporan técnicas adicionales. Tran *et al.* [19] proponen evaluación vectorizada en GP, logrando aceleraciones de 100x —una idea que nuestro sistema también implementa. López de Prado [1, 11] formalizó las herramientas de validación contra sobreajuste que constituyen la columna vertebral de nuestro pipeline: CPCV, DSR y PBO.

MAP-Elites [12] ha demostrado su valor en robótica y diseño, pero su uso en finanzas cuantitativas es incipiente. Pugh *et al.* [16] argumentan que en dominios complejos, la diversidad de soluciones es tan valiosa como la optimalidad de la mejor —un principio que nuestros resultados de portafolio confirman.

Nuestro trabajo se diferencia en tres aspectos: la combinación de GE + MAP-Elites +

CMA-ES en un pipeline de búsqueda de dos fases (estructura y parámetros), la validación estadística múltiple integrada al proceso de descubrimiento, y la evaluación sobre múltiples criptoactivos con datos intradía, lo que permite analizar si los patrones descubiertos son específicos de un activo o reflejan regularidades más generales.

4. Metodología

4.1. La gramática de estrategias

El espacio de estrategias posibles está definido por una gramática BNF con cuatro componentes: dirección, condiciones de entrada, lógica combinatoria y parámetros de salida. Cada estrategia tiene la forma:

```
<estrategia> ::= <direccion> WHEN <regla_entrada> EXIT <salidas>
```

La dirección es LONG (apuesta al alza) o SHORT (apuesta a la baja). La regla de entrada combina entre una y tres condiciones:

```
<regla_entrada> ::= <condicion>
                    | <condicion> AND <condicion>
                    | <condicion> AND <condicion> AND <condicion>
                    | <condicion> OR <condicion>
                    | (<condicion> AND <condicion>) OR <condicion>
```

La gramática está deliberadamente sesgada hacia reglas multi-condición (hay más producciones con AND que con una sola condición), promoviendo reglas selectivas que filtran señales falsas.

4.1.1. Indicadores: dos familias tipadas

Un aspecto crucial del diseño es la separación de indicadores en dos familias con escalas compatibles, que solo pueden compararse dentro de la misma familia:

Osciladores (escala 0–100): RSI (Relative Strength Index), Stochastic K y D, ADX (Average Directional Index), MFI (Money Flow Index). Cada uno con períodos evolucionables ($\{7, 9, 14, 21\}$).

Indicadores normalizados (adimensionales, típicamente entre -3 y $+3$): Percent B (posición dentro de las Bandas de Bollinger), MACD normalizado por volatilidad, posición

relativa del precio respecto a su media móvil, Rate of Change, ratio de volumen, ancho de Bandas de Bollinger, y ATR como porcentaje del precio.

Esta separación tipada es fundamental: impide comparaciones sin sentido físico (como $RSI > MACD$, que mezclaría unidades incompatibles) y garantiza que cualquier condición generada por la gramática sea semánticamente coherente.

4.1.2. Parámetros de salida

Los parámetros que determinan cuándo cerrar una posición también son parte del genoma:

- **Take profit:** nivel de ganancia objetivo, en múltiplos de ATR ($\{1, 1.5, 2, 3, 4, 5, 6, 8\}$).
- **Stop loss:** nivel de pérdida máxima, en múltiplos de ATR ($\{0.5, 0.75, 1, 1.5, 2, 2.5, 3\}$).
- **Trailing stop:** un stop que se ajusta a favor cuando el precio se mueve favorablemente ($\{1, 1.5, 2, 2.5, 3, 4, 5\}$ múltiplos de ATR).

El ATR (Average True Range) mide la volatilidad reciente del mercado, por lo que expresar los niveles de salida en unidades de ATR los hace *adaptativos*: en períodos de alta volatilidad, los stops se amplían automáticamente; en períodos tranquilos, se estrechan.

El espacio combinatorio resultante —considerando todos los indicadores, parámetros, lógicas y salidas posibles— es del orden de 10^8 estrategias distintas.

4.2. Evaluación vectorizada

Explorar un espacio de 10^8 posibilidades requiere evaluar cada candidata rápidamente. El sistema computa todos los indicadores técnicos como vectores completos (series temporales precalculadas), evalúa las condiciones como operaciones booleanas sobre vectores, y combina los resultados con operaciones lógicas. Un sistema de caché evita recalcular indicadores idénticos.

El resultado: una estrategia se evalúa sobre 85.000 observaciones en menos de 1 milisegundo, lo que permite evaluar decenas de miles de estrategias en tiempo razonable.

4.3. Motor evolutivo

La búsqueda evolutiva utiliza tres subpoblaciones de 100 individuos cada una, totalizando 300 estrategias por generación. Cada subpoblación emplea un criterio diferente para elegir qué individuos se reproducen:

- La primera favorece a los individuos con mejor rendimiento global (*selección por torneo*: se eligen 3 individuos al azar y se selecciona el mejor).
- La segunda favorece a los especialistas en objetivos específicos (*selección lexicográfica*: se filtra secuencialmente por cada métrica).
- La tercera elige padres al azar, priorizando la exploración pura.

Periódicamente, los mejores individuos *migran* entre subpoblaciones, compartiendo soluciones prometedoras. También se inyectan estrategias desde el archivo MAP-Elites, introduciendo diversidad comportamental.

Operadores genéticos. El crossover corta dos genomas en un punto aleatorio e intercambia las mitades. La mutación modifica cada codón con probabilidad 0,15: en el 60 % de los casos ajusta el valor en ± 1 (cambio gradual), en el 30 % lo reemplaza por un valor completamente aleatorio (cambio estructural), y en el 10 % intercambia dos codones vecinos.

Función de aptitud. La aptitud combina múltiples criterios en un único score:

$$f = \text{Sortino} + 10 \cdot \max(\text{CAGR}, 0) + 0,3 \cdot \min(\text{Calmar}, 5) + \min(\text{PF} - 1, 3) - 0,01 \cdot n_{\text{condiciones}} \quad (9)$$

El último término es una **presión de parsimonia**: penaliza la complejidad, favoreciendo estrategias más simples que son menos propensas al sobreajuste.

Se descartan automáticamente las estrategias con menos de 30 operaciones (evidencia estadística insuficiente), drawdown superior al 30 %, o expectativa negativa.

Rotación de ventanas temporales. Cada generación, la evaluación se realiza sobre 12 ventanas temporales de ~ 2 meses muestreadas aleatoriamente dentro del período de evolución. **Cada generación usa ventanas diferentes.** Esto obliga a las estrategias a rendir bien en períodos variados, no solo en uno particular —un mecanismo anti-sobreajuste activo durante la búsqueda.

4.4. Refinamiento local con CMA-ES

La Evolución Gramatical es eficaz para descubrir *estructuras* prometedoras —qué indicadores combinar, con qué lógica, en qué dirección— pero los parámetros numéricos que descubre están restringidos a los valores discretos definidos en la gramática (por ejemplo, períodos $\in \{7, 9, 14, 21\}$). Una vez que GE identifica una buena estructura, es natural

preguntarse: ¿existen valores de parámetros *entre* los disponibles que mejoren el rendimiento?

Para responder esta pregunta, incorporamos una segunda fase de optimización usando CMA-ES (*Covariance Matrix Adaptation Evolution Strategy*) [8]. CMA-ES es un algoritmo de optimización continua que adapta iterativamente una distribución gaussiana multivariada para modelar las correlaciones entre parámetros. A diferencia de un optimizador de gradiente, no requiere derivadas y maneja bien los paisajes de aptitud ruidosos y multimodales que caracterizan al backtesting.

El procedimiento de refinamiento funciona así:

1. Se extrae de cada estrategia validada el vector de parámetros continuos (períodos de indicadores, umbrales de comparación, múltiplos de ATR para salidas). Los elementos estructurales (qué indicadores, operadores lógicos) permanecen fijos.
2. Se define una región de búsqueda acotada alrededor de los valores descubiertos por GE ($\pm 50\%$ del valor original), para evitar que CMA-ES derive hacia zonas sin relación con la estructura descubierta.
3. Se ejecuta CMA-ES tres veces con semillas distintas, promediando los mejores parámetros encontrados (*consenso multi-semilla*). Esto regulariza el resultado y reduce la dependencia de la inicialización.
4. Los parámetros optimizados se validan con CPCV sobre los mismos folds que la estrategia original. Solo se aceptan si el rendimiento fuera de muestra mejora y la cantidad de operaciones no se reduce más de un 40 %.

Este enfoque en dos fases —GE para estructura, CMA-ES para parámetros— separa la búsqueda discreta de la continua, aprovechando las fortalezas de cada algoritmo.

4.5. Pipeline de validación

Las estrategias que sobreviven la evolución se someten a tres tests estadísticos independientes:

1. **CPCV** con $N = 10$ grupos, generando 252 evaluaciones fuera de muestra, con purge de 24 horas y embargo de 12 horas en las fronteras temporales.
2. **PBO**: fracción de evaluaciones CPCV con Sortino ≤ 0 . Criterio: $PBO < 0,50$.
3. **Permutación de señales**: 1.000 permutaciones aleatorias. Criterio: $p < 0,05$.

Una estrategia pasa la validación si cumple **los tres criterios simultáneamente**.

4.6. Validación walk-forward

Además de los tests estadísticos, se aplicó validación *walk-forward* para evaluar la estabilidad temporal de las estrategias. Este método simula cómo se habría comportado la estrategia si se hubiera desplegado en tiempo real:

1. Se divide el período completo en ventanas solapadas de 6 meses de entrenamiento seguidos de 2 meses de evaluación, con un paso de 1 mes.
2. En cada ventana, la estrategia (ya fija —no se re-optimiza) se evalúa sobre los 2 meses de evaluación, que son estrictamente posteriores al entrenamiento.
3. Se registra el retorno y el drawdown de cada ventana de evaluación.

Una estrategia robusta debería mostrar retornos positivos en la mayoría de las ventanas, no solo en el período OTS completo. Una estrategia que gana mucho en un período y pierde en todos los demás es frágil, aunque su resultado agregado sea positivo. Reportamos la **consistencia** (fracción de ventanas positivas) como métrica complementaria.

4.7. Evaluación final sobre datos reservados

Los últimos seis meses de datos (junio a noviembre 2025) se mantuvieron aislados durante todo el proceso de evolución y validación. Solo se usaron una única vez, al final, para evaluar las estrategias que pasaron todos los tests. No hubo iteración ni ajuste posterior: los resultados se reportan tal cual.

5. Configuración Experimental

5.1. Datos

Tabla 1: Datos utilizados. Los tres activos comparten la misma ventana temporal y resolución.

Parámetro	Valor
Instrumentos	BTC/USDT, ETH/USDT, BNB/USDT (futuros perpetuos, Binance)
Resolución temporal	15 minutos
Período de evolución	Enero 2023 – Mayo 2025 (~84.000 barras por activo)
Período de evaluación final	Junio 2025 – Noviembre 2025 (~17.000 barras)
Comisiones modeladas	0,01 % por lado (taker fee)
Slippage modelado	0,01 % por lado

La evolución se ejecutó de forma independiente para cada activo, utilizando la misma gramática y los mismos parámetros evolutivos. Esto permite evaluar si el framework descubre patrones en activos con microestructuras diferentes: BTC (el activo más líquido y

eficiente del ecosistema cripto), ETH (segundo en liquidez, con dinámica propia ligada al ecosistema DeFi) y BNB (menor liquidez, ligado al exchange Binance).

5.2. Parámetros de evolución

Tabla 2: Configuración del motor evolutivo.

Parámetro	Valor
Población total	300 (3 subpoblaciones de 100)
Generaciones máximas	150
Paciencia	25 generaciones sin mejora
Longitud del genoma	50 codones
Tasa de mutación	0,15 por codón
Tasa de crossover	0,70
Elitismo	3 %
Ventanas por generación	12
Duración de ventana	5.760 barras (~ 2 meses)
Tamaño de migración	3 individuos cada 15 generaciones

5.3. Ejecuciones

Para BTC se realizaron **seis ejecuciones independientes** con semillas distintas (42, 123, 314, 777, 999, 2024). Para ETH y BNB se realizaron tres ejecuciones cada uno (42, 123, 777). Cada ejecución parte de una población aleatoria diferente y converge de forma independiente, lo que permite evaluar la robustez de los resultados frente a la inicialización. Las estrategias descubiertas en ETH y BNB se sometieron adicionalmente a la fase de refinamiento CMA-ES.

6. Resultados

6.1. Evolución

Las seis ejecuciones convergieron entre las generaciones 44 y 73, generando entre 11.961 y 19.847 fenotipos únicos cada una. La Figura 1 muestra la progresión de la aptitud para una de las ejecuciones.

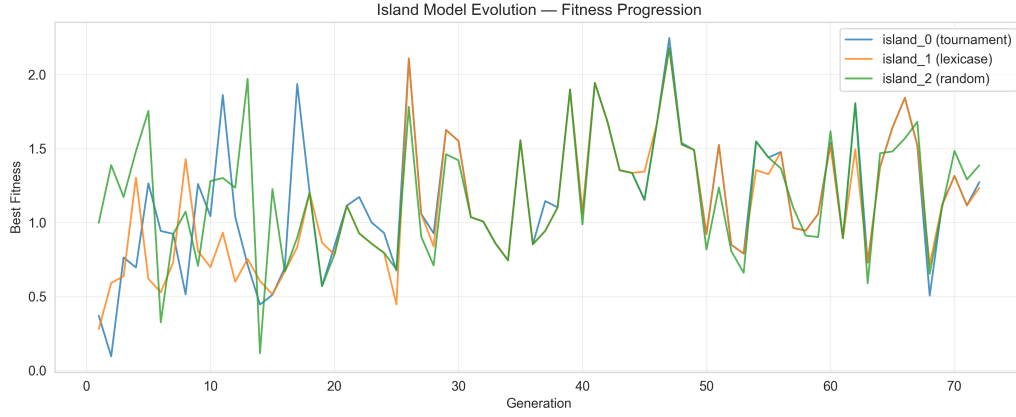


Figura 1: Progresión de la aptitud a lo largo de las generaciones. La aptitud mejora de $\sim 3,5$ en la generación 0 a $\sim 10,8$ en la generación 49 (un incremento de $3x$). Las tres subpoblaciones mantienen trayectorias diferentes, lo que confirma que las distintas presiones de selección inducen dinámicas evolutivas complementarias.

El archivo MAP-Elites alcanzó una ocupación de 57–60 % (26–27 de 45 celdas), con distribución equilibrada entre los tres regímenes de mercado (Tabla 3).

Tabla 3: Cobertura del archivo MAP-Elites por ejecución.

Semilla	Celdas ocupadas	Cobertura	Alcista	Bajista	Lateral
123	26/45	57,8 %	8	9	9
42	27/45	60,0 %	9	9	9
777	26/45	57,8 %	9	8	9

6.2. Estrategias descubiertas

En total, las seis ejecuciones produjeron 166 estrategias candidatas. De estas, **87 pasaron los tres criterios de validación** simultáneamente ($PBO < 0,50$, permutación $p < 0,05$, test t con $p < 0,05$).

La Tabla 4 muestra las 10 mejores estrategias por CAGR en el período de evaluación final.

Tabla 4: Top 10 estrategias en el período de evaluación final (junio–noviembre 2025).

#	Tipo	CAGR	Sortino	Max DD	PF	PBO
1	Contrarian bajista	20,9 %	0,656	−3,6 %	1,98	0,06
2	Momentum alcista	18,9 %	0,373	−8,7 %	1,37	0,00
3	Mean-reversion bajista	18,1 %	0,850	−2,3 %	1,68	0,22
4	Momentum alcista	17,7 %	0,343	−8,7 %	1,38	0,00
5	Flujo + momentum	11,2 %	0,140	−5,8 %	1,57	0,06
6	Confluencia multi-factor	9,9 %	0,089	−1,6 %	3,83	0,00
7	Posición + tendencia	8,0 %	0,269	−2,8 %	1,96	0,23
8	Flujo + oscilador	7,6 %	0,244	−3,8 %	1,23	0,10
9	Posición + volatilidad	6,6 %	0,242	−1,7 %	2,20	0,05
10	Multi-factor + momentum	5,7 %	0,111	−6,7 %	1,14	0,14

Las estrategias exhiben una variedad de enfoques que la gramática permitió descubrir:

- **Estrategias contrarian:** detectan condiciones extremas en osciladores (por ejemplo, RSI en zona de sobreventa) combinadas con filtros de volatilidad y volumen, apostando a que el movimiento extremo se revertirá parcialmente. Estas tienden a tener pocas operaciones pero alto factor de beneficio.
- **Estrategias de momentum:** identifican cruces entre indicadores de diferente velocidad (por ejemplo, un oscilador rápido cruzando sobre uno lento), entrando en la dirección del impulso. Tienden a operar con mayor frecuencia.
- **Estrategias multi-factor:** combinan tres condiciones de familias distintas (flujo de dinero, volatilidad, osciladores), entrando solo cuando todos los filtros coinciden. Son altamente selectivas y muestran los menores drawdowns.

6.3. Validación estadística

La Tabla 5 resume los resultados de validación para las estrategias aprobadas.

Tabla 5: Resumen de validación. Las 87 estrategias aprobadas cumplieron simultáneamente los tres criterios.

Métrica	Rango	Criterio
PBO	0,000 – 0,460	< 0,50
Permutación p	0,000 – 0,041	< 0,05
CPCV Sortino medio	0,037 – 0,342	> 0
% splits positivos	53 % – 100 %	—

Cabe destacar que 16 estrategias obtuvieron un PBO de exactamente 0,000, lo que significa que las 252 evaluaciones CPCV fueron positivas —una señal particularmente fuerte de

consistencia fuera de muestra.

6.4. Construcción de portafolios

Aunque las estrategias individuales muestran rendimientos positivos, la verdadera fortaleza del enfoque surge al combinar múltiples estrategias decorrelacionadas en un portafolio. Se evaluaron cuatro configuraciones (Tabla 6).

Tabla 6: Portafolios en el período de evaluación final. BTC perdió 31,6 % en el mismo período.

Portafolio	N	CAGR	Max DD	Sortino	Calmar
Agresivo	1	20,9 %	−3,6 %	0,21	5,8
Alto Retorno	3	20,0 %	−2,8 %	0,53	7,2
Ajustado por Riesgo	10	9,9 %	−0,8 %	1,51	11,6
Conservador	8	9,3 %	−0,4 %	0,84	21,9
Buy & Hold BTC	—	−31,6 %	−31,5 %	−1,14	−1,0

El portafolio “Ajustado por Riesgo” (10 estrategias con pesos optimizados) merece atención: logra un Sortino de 1,51 —indicando que el retorno obtenido compensa ampliamente el riesgo asumido— con un drawdown máximo de apenas 0,8 %. El portafolio “Conservador” alcanza un Calmar de 21,9, una relación retorno/drawdown excepcional.

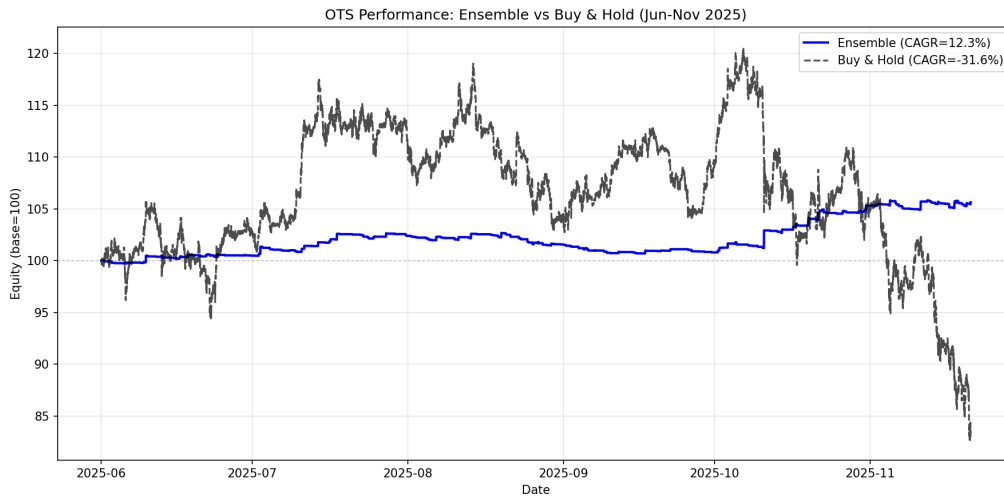


Figura 2: Evolución del capital del portafolio de 10 estrategias (línea ascendente) comparado con buy-and-hold de BTC (línea descendente) en el período de evaluación final. El portafolio crece de forma estable mientras BTC pierde un tercio de su valor.

La clave de este comportamiento es la **decorrelación** entre estrategias. La Figura 3 muestra que las correlaciones entre las 10 estrategias son mayoritariamente bajas, lo que permite que las ganancias de unas compensen las pérdidas transitorias de otras.

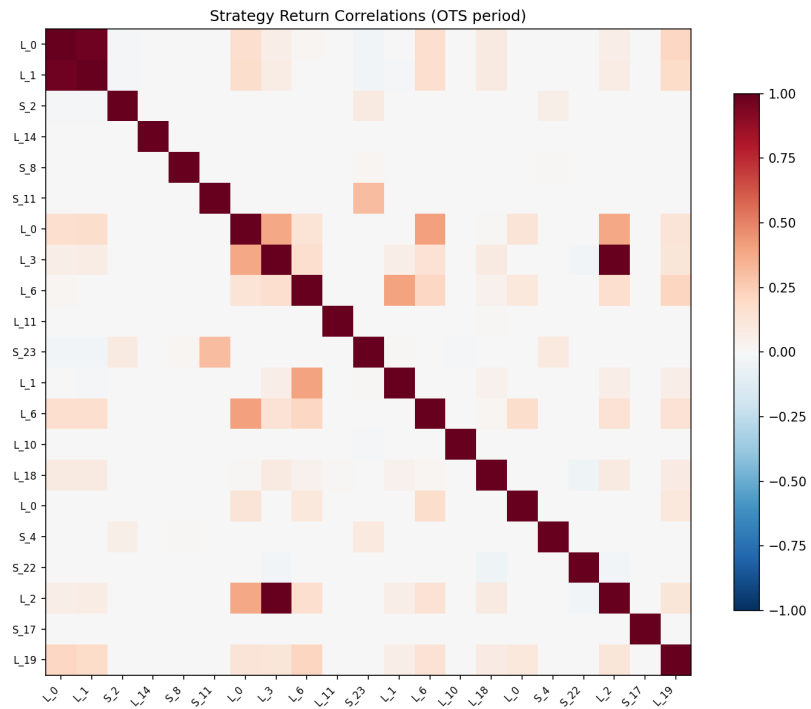


Figura 3: Matriz de correlación entre las estrategias del portafolio. Las correlaciones son mayoritariamente inferiores a 0,3, lo que explica la reducción de drawdown observada en el portafolio combinado.

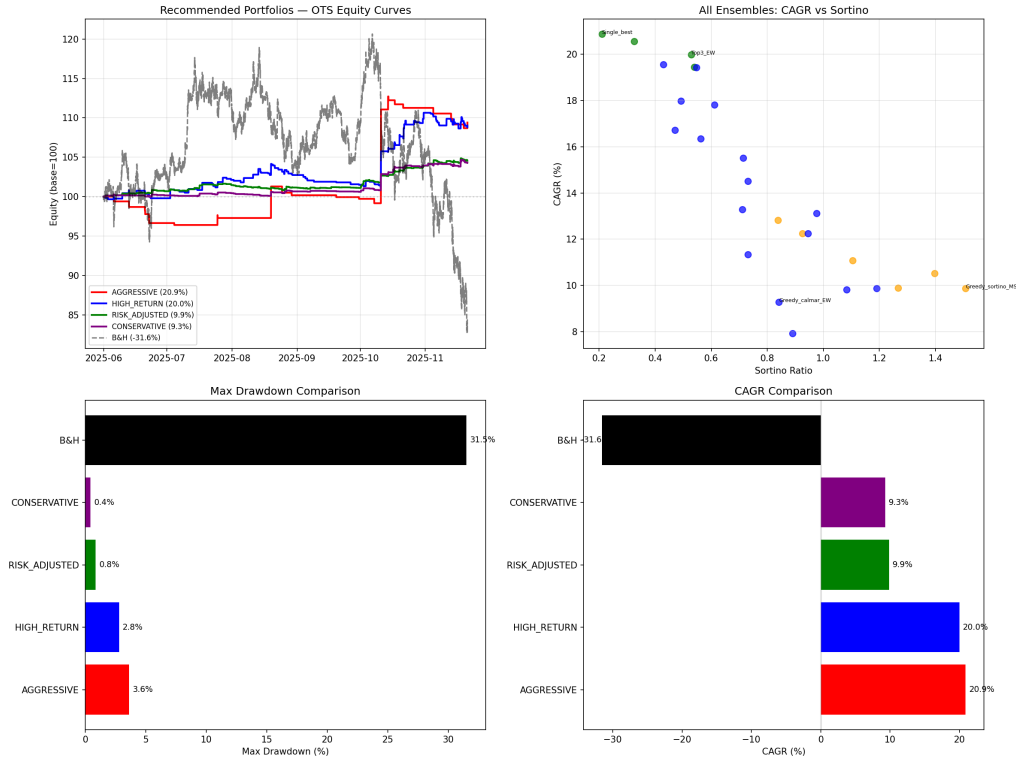


Figura 4: Frontera de eficiencia de los portafolios evaluados. El eje horizontal muestra el drawdown máximo y el vertical el retorno anualizado. Los cuatro portafolios recomendados trazan una frontera desde alta rentabilidad (mayor riesgo) hasta mínimo drawdown (menor rentabilidad).

6.5. Extensión multi-activo

Para evaluar si el framework generaliza más allá de BTC, se ejecutó el mismo pipeline sobre ETH/USDT y BNB/USDT. Los resultados muestran que la Evolución Gramatical descubre estrategias validadas en los tres activos, aunque con características diferentes.

Optimización CMA-ES por activo. La segunda fase de optimización reveló un hallazgo inesperado: el beneficio de CMA-ES depende fuertemente del activo (Tabla 7).

Tabla 7: Efecto de la optimización CMA-ES por activo. Se comparan las estrategias originales (GE) con las refinadas (GE + CMA-ES).

Activo	Mejoran	Empeoran	Sin cambio	Recomendación
BTC	0/15	8/15	7/15	No optimizar
ETH	4/13	1/13	8/13	Optimizar
BNB	5/15	6/15	4/15	Optimizar (selectivamente)

En BTC, ninguna estrategia mejoró con CMA-ES y varias empeoraron. Una interpretación posible es que el paisaje de aptitud de BTC —el activo más líquido y arbitrado del

ecosistema— tiene óptimos estrechos y aislados: los parámetros discretos de la gramática ya capturan lo mejor disponible, y cualquier perturbación continua degrada el rendimiento. En contraste, ETH y BNB parecen tener paisajes más suaves donde existe margen para el refinamiento. Por ejemplo, una estrategia de BNB pasó de un CAGR de 11,7 % a 64,6 % tras CMA-ES, con el PBO cayendo de 0,24 a 0,004 —una mejora notable tanto en rendimiento como en robustez estadística.

Portafolio multi-activo. Combinando las mejores estrategias de cada activo —originales para BTC, optimizadas con CMA-ES para ETH y BNB— se construyó un portafolio diversificado de 10 estrategias (Tabla 8).

Tabla 8: Portafolio multi-activo: 10 estrategias seleccionadas para evaluación final. Las estrategias marcadas con * fueron refinadas con CMA-ES.

Activo	Dirección	Tipo	CAGR (OTS)	Max DD (OTS)
BTC	SHORT	GE	36,0 %	−4,7 %
BTC	SHORT	GE	31,5 %	−3,6 %
BTC	LONG	GE	15,3 %	−14,0 %
ETH	LONG	GE	27,6 %	−5,8 %
ETH	SHORT	GE	19,9 %	−3,2 %
ETH	SHORT	GE + CMA-ES*	31,3 %	−2,1 %
BNB	LONG	GE + CMA-ES*	41,3 %	−6,4 %
BNB	SHORT	GE	21,4 %	−4,8 %
BNB	LONG	GE + CMA-ES*	64,6 %	−8,2 %
BNB	LONG	GE + CMA-ES*	48,6 %	−5,1 %

6.6. Estabilidad walk-forward

La validación walk-forward evaluó cada portafolio sobre 10 ventanas temporales consecutivas, simulando un despliegue progresivo en el tiempo (Tabla 9).

Tabla 9: Validación walk-forward por activo (10 ventanas, estrategias con filtro de régimen). B&H indica buy-and-hold del activo subyacente en el mismo período.

Activo	Ventanas positivas	Retorno compuesto	B&H compuesto	Peor ventana
BTC	9/10 (90 %)	+251 %	+188 %	−5,0 %
ETH	9/10 (90 %)	+92,5 %	+52,9 %	−4,2 %
BNB	8/10 (80 %)	+59,6 %	+270 %	−6,1 %

Dos observaciones merecen atención. Primero, BTC y ETH superan al buy-and-hold tanto en retorno como en consistencia, mientras que BNB logra retornos inferiores al subyacente pero con un perfil de riesgo muy diferente: la peor ventana individual es −6,1 % frente a

caídas superiores al 30 % en el buy-and-hold de BNB durante el mismo período. Segundo, la consistencia de 80–90 % de ventanas positivas sugiere que las estrategias capturan patrones que persisten a lo largo del tiempo, no solo en el período OTS específico.

7. Discusión

7.1. El rol de la diversificación

Los resultados muestran que la combinación de estrategias decorrelacionadas produce perfiles de riesgo-retorno sustancialmente mejores que cualquier estrategia individual. Esto es consistente con la teoría clásica de portafolios: cuando los activos (en este caso, las curvas de equity de distintas estrategias) tienen correlación baja, el riesgo del conjunto se reduce sin sacrificar proporcionalmente el retorno.

En este contexto, MAP-Elites cumple un rol que va más allá de la exploración evolutiva: al forzar la diversidad comportamental, produce los ingredientes necesarios para construir portafolios robustos. Una búsqueda que convergiera a una sola “mejor” estrategia no permitiría esta construcción.

7.2. Interpretabilidad

Una ventaja significativa de la Evolución Gramatical frente a enfoques de caja negra (redes neuronales, reinforcement learning) es que las estrategias descubiertas son *legibles*. Cada regla puede expresarse como una combinación de condiciones sobre indicadores técnicos conocidos, lo que permite:

- Verificar que la lógica tiene sentido económico (no es un artefacto numérico).
- Comunicar la estrategia a otros investigadores o profesionales.
- Diagnosticar fallos cuando las condiciones de mercado cambian.

Las estrategias descubiertas emplean combinaciones no triviales de indicadores —por ejemplo, combinar un filtro de sobreventa (RSI) con uno de expansión de volatilidad (ancho de Bollinger) y uno de volumen— que difícilmente un operador humano diseñaría manualmente, pero que resultan interpretables una vez descubiertas.

7.3. Eficiencia de mercado y capacidad predictiva individual

Los resultados del test de White (Reality Check) sobre las estrategias individuales de BTC arrojan p -valores de 0,34 frente al buy-and-hold, lo cual no permite rechazar la hipótesis nula de que ninguna estrategia individual supera al benchmark de forma estadísticamente

significativa. Este resultado es consistente con la hipótesis de mercados eficientes en su forma débil: BTC, siendo el activo más líquido del ecosistema, deja poco margen para señales técnicas individuales.

Sin embargo, el portafolio diversificado sí muestra un perfil de riesgo-retorno favorable, lo que sugiere que el valor no reside en estrategias individuales sino en la **combinación de reglas decorrelacionadas**. Individualmente, cada regla captura una señal débil; colectivamente, las señales se refuerzan mientras los errores se cancelan.

7.4. La asimetría CMA-ES entre activos

El hallazgo de que CMA-ES mejora ETH y BNB pero no BTC tiene implicaciones teóricas interesantes. BTC, por su mayor liquidez y volumen de arbitraje, podría tener un paisaje de aptitud con óptimos más estrechos y aislados: los parámetros discretos de la gramática ya se encuentran cerca del mejor rendimiento alcanzable, y cualquier perturbación continua desestabiliza la estrategia. En activos menos eficientes, el paisaje parece más suave, con gradientes que CMA-ES puede seguir para encontrar mejores configuraciones. Esta observación sugiere que la decisión de aplicar o no optimización local debería ser adaptativa al activo.

7.5. Limitaciones

1. **Indicadores técnicos estándar:** la gramática utiliza osciladores y ratios normalizados basados en precio y volumen. Datos alternativos (order flow, liquidaciones, métricas on-chain) podrían proveer señales complementarias.
2. **Una sola resolución temporal:** 15 minutos. Resoluciones mayores (1h, 4h, diaria) podrían capturar patrones de naturaleza diferente.
3. **Período de evaluación específico:** junio–noviembre 2025 fue un mercado predominantemente bajista. La validez de los resultados en otros entornos de mercado requiere investigación adicional.
4. **Costos simplificados:** se modelaron comisiones y slippage fijos; el impacto de mercado real podría diferir para volúmenes grandes.
5. **Significancia estadística individual:** ninguna estrategia individual supera el test de White al 5 %. El valor del framework reside en la generación de diversidad para portafolios, no en estrategias individuales dominantes.

8. Conclusiones y Trabajo Futuro

8.1. Conclusiones

Este trabajo presentó un framework de descubrimiento automatizado de estrategias cuantitativas que combina Evolución Gramatical, MAP-Elites, optimización local con CMA-ES y validación estadística rigurosa. Los resultados principales son:

1. La Evolución Gramatical demostró ser un mecanismo efectivo para explorar espacios combinatorios de $\sim 10^8$ reglas cuantitativas, descubriendo estrategias interpretables con parámetros adaptativos. La búsqueda guiada por aptitud evaluó menos del 0,03 % del espacio total, pero concentrada en regiones de alta calidad —algo que la fuerza bruta o el muestreo aleatorio no pueden lograr.
2. MAP-Elites produjo repertorios diversos de estrategias organizadas por frecuencia, complejidad y régimen de mercado, alcanzando coberturas del 58–60 % del espacio comportamental definido. Esta diversidad resultó ser el ingrediente clave para la construcción de portafolios robustos.
3. La optimización en dos fases (GE para estructura, CMA-ES para parámetros) mostró beneficios asimétricos entre activos: mejoras significativas en ETH y BNB, ninguna mejora en BTC. Este hallazgo sugiere que la rugosidad del paisaje de aptitud varía con la eficiencia del mercado.
4. El pipeline de validación (CPCV + PBO + permutación) filtró eficazmente las estrategias sobreajustadas: de 166 candidatas en BTC, 87 superaron los tres criterios simultáneamente. Los tests de White confirman que ninguna estrategia individual alcanza significancia estadística, pero los portafolios diversificados muestran perfiles de riesgo-retorno consistentemente favorables.
5. El framework generalizó a tres activos con microestructuras diferentes. La validación walk-forward mostró consistencia del 80–90 % de ventanas positivas en BTC, ETH y BNB, con un portafolio multi-activo de 10 estrategias que obtuvo un retorno anualizado de 9,9 % y drawdown máximo de $-0,8$ % durante un mercado bajista.

8.2. Trabajo futuro

Varias extensiones podrían ampliar el alcance de los resultados:

- **Aprendizaje por refuerzo:** usar las estrategias descubiertas como política inicial para un agente de RL, combinando la interpretabilidad de GE con la capacidad adaptativa del aprendizaje por refuerzo.

- **Señales multi-resolución:** combinar indicadores calculados sobre velas de 15 minutos, 1 hora y 4 horas para capturar patrones jerárquicos. La gramática actual soporta esta extensión, pero los experimentos presentados se limitan a la resolución de 15 minutos.
- **Datos alternativos:** incorporar métricas de flujo de órdenes, tasas de financiamiento e indicadores on-chain como terminales adicionales en la gramática.
- **Position sizing adaptativo:** evolucionar reglas de tamaño de posición como parte del genoma, utilizando señales de régimen de volatilidad (por ejemplo, modelos HMM) para modular la exposición.
- **Co-evolución competitiva:** evolucionar poblaciones de estrategias que compiten entre sí, simulando la dinámica de un mercado con múltiples participantes informados.

Referencias

- [1] David H. Bailey and Marcos López de Prado. The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *Journal of Portfolio Management*, 40(5):94–107, 2014.
- [2] David H. Bailey, Jonathan Borwein, and Marcos López de Prado. A note on the use of cross-validation in finance. *Journal of Machine Learning Research*, 2016.
- [3] David H. Bailey, Jonathan M. Borwein, Marcos López de Prado, and Qiji Jim Zhu. The probability of backtest overfitting. *Journal of Computational Finance*, 20(4):39–69, 2017.
- [4] Anthony Brabazon and Michael O’Neill. Biologically inspired algorithms for financial modelling. In *Natural Computing Series*. Springer, 2006.
- [5] Shu-Heng Chen, Chia-Hsuan Yeh, et al. Genetic programming for financial trading rule discovery. *Computational Economics*, 2002.
- [6] Eugene F. Fama. Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2):383–417, 1970.
- [7] David E. Goldberg. Genetic algorithms in search, optimization, and machine learning. 1989.
- [8] Nikolaus Hansen, Sibylle D. Müller, and Petros Koumoutsakos. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES). *Evolutionary Computation*, 11(1):1–18, 2003.

- [9] John H. Holland. *Adaptation in natural and artificial systems*. 1975.
- [10] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.
- [11] Marcos López de Prado. *Advances in financial machine learning*. 2018.
- [12] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [13] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.
- [14] Michael O’Neill and Conor Ryan. Grammatical evolution: Evolutionary automatic programming in an arbitrary language. *Genetic Programming and Evolvable Machines*, 4(4):349–370, 2003.
- [15] Jean-Yves Potvin, Patrick Soriano, and Maxime Vallée. Generating trading rules on the stock markets with genetic programming. *Computers & Operations Research*, 31(7):1033–1047, 2004.
- [16] Justin K. Pugh, Lisa B. Soros, and Kenneth O. Stanley. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, 3:40, 2016.
- [17] William F. Sharpe. Mutual fund performance. *The Journal of Business*, 39(1):119–138, 1966.
- [18] Frank A. Sortino and Robert van der Meer. Performance measurement in a downside risk framework. *The Journal of Investing*, 3(3):59–64, 1994.
- [19] Binh Tran et al. Vectorial genetic programming for efficient financial strategy discovery. *arXiv preprint*, 2025.
- [20] Halbert White. A reality check for data snooping. *Econometrica*, 68(5):1097–1126, 2000.
- [21] Terry W. Young. Calmar ratio: A smoother tool. *Futures*, 20(1):40, 1991.